

PROJECT REPORT

PROGRAMMING CALCULATOR USING PYTHON

A Python-based calculator for arithmetic, number-system conversion, and bitwise programming operations.

Name: _____

Roll No.: _____

Registration No.: _____

Course: _____

Subject: _____

Institution: _____

Academic Year: 2026–2027

1. Introduction

A programming calculator is a computational tool designed to perform mathematical, numerical, and bitwise operations commonly used in computer programming. Unlike a conventional calculator, it works with Decimal, Binary, Octal, and Hexadecimal number systems and provides AND, OR, XOR, NOT, left shift, and right shift operations.

The project is implemented in Python with a simple menu-driven interface. It demonstrates functions, loops, conditional statements, exception handling, number-system conversion, bitwise operators, input validation, and modular organization.

2. Problem Statement

Students learning programming frequently need to perform calculations involving different number systems and bitwise operations. Using separate tools for these tasks can be inconvenient. This project combines conversion, arithmetic, bitwise operations, input validation, and error handling into one Python application.

3. Objectives

- Develop a programming calculator using Python.
- Perform arithmetic calculations efficiently.
- Convert numbers between decimal, binary, octal, and hexadecimal systems.
- Implement bitwise AND, OR, XOR, NOT, left shift, and right shift operations.
- Provide input validation and error handling.
- Demonstrate practical Python programming concepts.
- Provide an easy-to-use menu-driven interface.

4. Scope of the Project

Number System Operations

- Decimal to Binary
- Decimal to Octal
- Decimal to Hexadecimal
- Binary to Decimal
- Octal to Decimal
- Hexadecimal to Decimal

Arithmetic Operations

- Addition
- Subtraction

- Multiplication
- Integer Division
- Modulus

Bitwise Operations

- Bitwise AND
- Bitwise OR
- Bitwise XOR
- Bitwise NOT
- Left Shift
- Right Shift

Error Handling

- Invalid numerical input
- Invalid binary, octal, or hexadecimal input
- Division by zero
- Modulus by zero
- Invalid menu choices
- Negative shift positions

5. Functional Requirements

5.1 Number System Conversion Module

Accepts a number, validates it according to its base, converts it using Python integer conversion facilities, and displays the result.

5.2 Arithmetic Operations Module

Performs addition, subtraction, multiplication, integer division, and modulus on integer values.

5.3 Bitwise Operations Module

Performs AND, OR, XOR, NOT, left shift, and right shift operations on integers.

5.4 Input Validation Module

Checks user input before processing. Binary values, for example, are restricted to digits 0 and 1.

5.5 Error Handling Module

Handles invalid input and mathematical errors such as division by zero without terminating the application.

6. Non-Functional Requirements

Requirement	Description
Performance	Calculations are performed locally and return results immediately.
Usability	A numbered menu provides a simple interaction model.
Reliability	Validation and exception handling reduce failures caused by invalid input.
Maintainability	Operations are separated into functions, making the program easier to modify.
Resource Efficiency	The program uses minimal system resources and requires no database or network service.
Error Handling	Common input and mathematical errors are detected and reported.

7. System Architecture

The current project follows a simple procedural architecture:

```
USER → MAIN MENU → SELECT OPERATION → INPUT → VALIDATION → PROCESSING → RESULT → RETURN
TO MENU
```

The main controller displays the menu and directs execution to conversion, arithmetic, or bitwise functions. Input-handling functions validate values before operations are performed.

8. Workflow

```
START
↓
Display Main Menu
↓
User Selects Operation
↓
Take Input
↓
Validate Input
↓
Perform Operation
↓
Display Result
↓
Return to Menu or Exit
↓
END
```

9. Use Case Description

Primary Actor: User

- Select an operation from the main menu.
- Enter numerical input.
- Convert between number systems.
- Perform arithmetic calculations.
- Perform bitwise calculations.

- View the result.
- Exit the application.

10. Program Components

The current implementation is function-based. Its major logical components are:

```

Programming Calculator
■■■ Menu Management: show_menu()
■■■ Input Management: get_decimal(), get_binary(), get_octal(), get_hexadecimal()
■■■ Conversion: decimal_to_binary(), decimal_to_octal(), decimal_to_hexadecimal(),
binary_to_decimal(), octal_to_decimal(), hexadecimal_to_decimal()
■■■ Arithmetic: addition(), subtraction(), multiplication(), integer_division(), modulus()
■■■ Bitwise: bitwise_and(), bitwise_or(), bitwise_xor(), bitwise_not(), left_shift(),
right_shift()
■■■ Main Controller: main()

```

11. Algorithms Used

11.1 Decimal to Binary

Python's `bin()` function is used to convert an integer to binary notation.

Example: `bin(10) → 0b1010`

11.2 Decimal to Octal

Example: `oct(10) → 0o12`

11.3 Decimal to Hexadecimal

Example: `hex(255) → 0xff`

11.4 Base to Decimal

The `int(value, base)` function is used with base 2 for binary, base 8 for octal, and base 16 for hexadecimal.

12. Bitwise Operators

Operator	Operation	Example	Meaning
&	AND	12 & 10	Sets a bit when both corresponding bits are 1.
	OR	12 10	Sets a bit when either corresponding bit is 1.
^	XOR	12 ^ 10	Sets a bit when corresponding bits are different.
~	NOT	~12	Inverts the bits of the integer.
<<	Left Shift	12 << 2	Shifts bits to the left.
>>	Right Shift	12 >> 2	Shifts bits to the right.

Example: 12 = 1100 and 10 = 1010. Their bitwise AND is 1000, which equals 8.

13. Design Decisions and Rationale

Choice of Python

Python was selected because it has simple syntax, built-in integer operations, number conversion functions, bitwise operators, and straightforward exception handling.

Menu-Based Interface

A menu-driven interface was selected because it is simple to operate and clearly separates the available calculator functions.

Function-Based Design

Separate functions were created for different operations so that each operation can be understood, maintained, and tested independently.

14. Implementation Details

- Functions encapsulate individual operations.
- Conditional statements handle menu selection.
- A while loop keeps the calculator running until Exit is selected.
- try-except blocks handle invalid numerical inputs.
- Python's built-in arithmetic and bitwise operators perform calculations.
- Input validation prevents invalid values from being processed.

15. Testing

Test Case	Input	Expected Output	Status
Decimal to Binary	10	1010	Pass
Decimal to Hexadecimal	255	FF	Pass
Binary to Decimal	1010	10	Pass
Addition	25, 15	40	Pass
Bitwise AND	12, 10	8	Pass
Bitwise OR	12, 10	14	Pass
Bitwise XOR	12, 10	6	Pass
Division by Zero	10, 0	Error message	Pass
Invalid Binary	10201	Invalid binary number	Pass
Invalid Menu Choice	25	Invalid choice	Pass

16. Sample Output

```
PROGRAMMING CALCULATOR
```

```
Number System Conversion
```

1. Decimal to Binary
2. Decimal to Octal
3. Decimal to Hexadecimal
4. Binary to Decimal
5. Octal to Decimal
6. Hexadecimal to Decimal

Arithmetic Operations

7. Addition
8. Subtraction
9. Multiplication
10. Integer Division
11. Modulus

Bitwise Operations

12. Bitwise AND
13. Bitwise OR
14. Bitwise XOR
15. Bitwise NOT
16. Left Shift
17. Right Shift

18. Exit

17. Challenges Faced

- Handling different number systems and their valid digit ranges.
- Validating user input before conversion.
- Preventing division and modulus by zero.
- Understanding signed integer behavior for bitwise NOT.
- Keeping the menu organized as features were added.

18. Learnings and Key Takeaways

- Python programming fundamentals
- Functions and modular organization
- Loops and conditional statements
- Exception handling
- Number-system representation
- Binary and bitwise operations
- Input validation
- Testing and debugging

19. Future Enhancements

- Graphical user interface using Tkinter.

- Calculation history.
- Scientific calculator functions.
- Floating-point support.
- Signed and unsigned integer modes.
- Two's complement representation.
- 8-bit, 16-bit, 32-bit, and 64-bit word-size modes.
- Binary arithmetic visualization.
- Automated unit testing.
- Modular multi-file project architecture.

20. Current Project Structure

```
Programming Calculator/
■■■■ programming_calculator.py
```

For a larger academic submission, the project can be reorganized into multiple Python modules such as `main.py`, `conversions.py`, `arithmetic.py`, `bitwise.py`, `validation.py`, `input_handler.py`, `calculator.py`, and `tests.py`.

21. Conclusion

The Programming Calculator is a Python-based computational application that combines arithmetic calculations, number-system conversions, and bitwise operations into a single program. It demonstrates practical use of Python functions, loops, conditional statements, exception handling, validation, and bitwise operators.

The application provides a simple way to perform programming-related calculations and understand the relationship between decimal, binary, octal, and hexadecimal representations. It can be extended with a graphical interface, history, additional programmer features, automated tests, and modular architecture.

22. References

- Python Documentation — Python programming language and built-in functions.
- Python documentation — Integer conversion and number-system functions.
- Python documentation — Bitwise operators.
- VITyarthi — Build Your Own Project: General Project Instructions & Submission Guidelines.